AWS Connect Integration Guide for the Dialer App

This document is written for a coding agent (codex/Claude/Cursor) that needs to wire the **Dialer to Life Insurance** web app (`index.html`) up to Amazon Connect so the "Call" button places a real outbound phone call.

The frontend already POSTs to `/api/call` with `{ phone, contactId }` when the user clicks Call. Your job is to build the backend that turns that request into a real Amazon Connect outbound voice contact.

---

1. What you are building

Two pieces:

1. AWS Connect instance - a virtual call center with one claimed phone number and one simple contact flow.
2. Backend API (Node.js Express or Python FastAPI) - a tiny server that exposes POST /api/call, validates input, and calls the Connect StartOutboundVoiceContact API.

End-to-end flow when the user clicks "Call":

```
Browser  --POST /api/call-->  Backend  --StartOutboundVoiceContact-->  AWS Connect
                                                                          |
                                                                          v
                                                                  Contact flow plays / connects
agent
                                                                          |
                                                                          v
                                                                  Customer's phone rings
```

---

2. AWS Connect setup (one-time, in the AWS Console)

2.1 Create a Connect instance

1. Open the AWS Console -> Amazon Connect -> Add an instance.
2. Identity management: Store users in Amazon Connect (simplest).
3. Administrator: Create one admin user (you will use this to log into the agent CCP).
4. Telephony: Check Allow outbound calls. (Inbound is optional.)
5. Data storage: Accept the auto-created S3 bucket for call recordings.

6. Review & launch. Provisioning takes ~5 minutes.

Save these values (you will need them in `.env`):

- Instance ID - the UUID at the end of the instance ARN. ARN looks like arn:aws:connect:us-east-1:123456789012:instance/<INSTANCE_ID>.
- Region - e.g. us-east-1.

2.2 Claim a phone number

1. In the Connect admin UI: Channels -> Phone numbers -> Claim a number.
2. Type: DID (Direct Inward Dial). Country: US. Pick any available number.
3. Description: "Dialer outbound caller ID".
4. Assign a contact flow later (we will create one next).
5. Save the number in E.164 format (e.g. +15558675309) - this is your SOURCE_PHONE_NUMBER.

2.3 Create a minimal outbound contact flow

1. Routing -> Contact flows -> Create contact flow. Name it Outbound-Simple.
2. Drag in these blocks and connect them in order:
   - Set voice -> Joanna (default is fine).
   - Play prompt -> text-to-speech: "Connecting your call. Please hold." (Optional, can be deleted to skip.)
   - Set working queue -> select BasicQueue (the default queue every instance has).
   - Transfer to queue -> end the flow here so a logged-in agent gets the call.
3. Save -> Publish.
4. Open the published flow and copy the Contact flow ID from the URL - it is the UUID after contact-flow/. Save as CONTACT_FLOW_ID.

***Alternative without an agent:** if you only want to place the call and play a recorded message (no human agent), replace `Transfer to queue` with `Play prompt` -> `Disconnect`. The customer will hear the prompt and the call ends. This is useful for testing the integration end-to-end before staffing an agent.*

2.4 Assign the queue to a routing profile (only if using `Transfer to queue`)

The admin user you created already has the "Admin" routing profile, which includes `BasicQueue`. No action needed unless you create more users.

2.5 Create an IAM user for the backend

1. AWS Console -> IAM -> Users -> Create user. Name: dialer-backend.
2. Attach policies directly -> create an inline policy with this JSON (replace the ARN with yours):

```
``json    {      "Version": "2012-10-17",      "Statement": [        {
```

```
"Effect": "Allow",          "Action": [
"connect:StartOutboundVoiceContact",            "connect:StopContact",
"connect:DescribeContact"          ],          "Resource":
"arn:aws:connect:us-east-1:123456789012:instance/INSTANCE_ID/*"        }      ]
}
```

3. After the user is created, open the user -> Security credentials -> Create access key -> Application running outside AWS.
4. Save AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY to .env. Do not commit them.

2.6 (Required for live agent audio) Set up a softphone

To actually hear/speak on the call from the browser, an agent must log into the Connect Contact Control Panel (CCP):

1. In Connect admin UI -> Users -> User management -> open your admin user -> Edit.
2. Set Phone type to Soft phone. Save.
3. Open https://<instance-alias>.my.connect.aws/ccp-v2/ in Chrome (allow microphone).
4. Set status to Available.

When the backend calls `StartOutboundVoiceContact` and the contact flow hits `Transfer to queue`, your CCP will ring - accept it, and you are now bridged to the customer.

If you want to embed the CCP inside the dialer page itself, follow §6 below ("Streams SDK").

---

3. Backend implementation

Pick one of the two stacks below. Both expose `POST /api/call` and accept the body the dialer already sends.

3.1 Node.js (Express) - recommended

Install:

```
mkdir backend && cd backend
npm init -y
npm install express cors dotenv @aws-sdk/client-connect
```

`backend/.env`:

```
AWS_REGION=us-east-1
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
CONNECT_INSTANCE_ID=00000000-0000-0000-0000-000000000000
CONNECT_CONTACT_FLOW_ID=00000000-0000-0000-0000-000000000000
```

```
SOURCE_PHONE_NUMBER=+15558675309
PORT=8787
```

`backend/server.js`:

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { ConnectClient, StartOutboundVoiceContactCommand } from '@aws-sdk/client-connect';

const app = express();
app.use(cors());
app.use(express.json());

const connect = new ConnectClient({ region: process.env.AWS_REGION });

function toE164(raw) {
  const digits = String(raw || '').replace(/\D/g, '');
  if (digits.length === 10) return '+1' + digits;
  if (digits.length === 11 && digits.startsWith('1')) return '+' + digits;
  if (raw && raw.startsWith('+')) return raw;
  return null;
}

app.post('/api/call', async (req, res) => {
  const { phone, contactId } = req.body || {};
  const destination = toE164(phone);
  if (!destination) return res.status(400).json({ error: 'Invalid phone number' });

  try {
    const out = await connect.send(new StartOutboundVoiceContactCommand({
      InstanceId: process.env.CONNECT_INSTANCE_ID,
      ContactFlowId: process.env.CONNECT_CONTACT_FLOW_ID,
      DestinationPhoneNumber: destination,
      SourcePhoneNumber: process.env.SOURCE_PHONE_NUMBER,
      Attributes: contactId ? { dialerContactId: String(contactId) } : undefined,
    }));
    res.json({ ok: true, contactId: out.ContactId });
  } catch (err) {
    console.error('Connect error:', err);
    res.status(502).json({ error: err.name || 'CallFailed', message: err.message });
  }
});

app.listen(process.env.PORT, () => console.log('Dialer backend on :' + process.env.PORT));
```

Run:

```
node server.js
```

Then in the browser, set `window.CALL_API = 'http://localhost:8787/api/call'` before `index.html` loads, OR proxy `/api/call` to `localhost:8787` from whatever serves the static HTML.

3.2 Python (FastAPI) alternative

Install:

```
pip install fastapi uvicorn boto3 python-dotenv
```

`backend/server.py`:

```
import os, re
import boto3
from dotenv import load_dotenv
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

load_dotenv()
app = FastAPI()
app.add_middleware(CORSMiddleware,        allow_origins=["*"],        allow_methods=["*"],
allow_headers=["*"])

connect = boto3.client("connect", region_name=os.environ["AWS_REGION"])

class CallReq(BaseModel):
    phone: str
    contactId: str | None = None

def to_e164(raw: str) -> str | None:
    digits = re.sub(r"\D", "", raw or "")
    if len(digits) == 10: return "+1" + digits
    if len(digits) == 11 and digits.startswith("1"): return "+" + digits
    if raw and raw.startswith("+"): return raw
    return None

@app.post("/api/call")
def place_call(req: CallReq):
    dest = to_e164(req.phone)
    if not dest:
        raise HTTPException(400, "Invalid phone number")
    try:
        resp = connect.start_outbound_voice_contact(
            InstanceId=os.environ["CONNECT_INSTANCE_ID"],
            ContactFlowId=os.environ["CONNECT_CONTACT_FLOW_ID"],
            DestinationPhoneNumber=dest,
            SourcePhoneNumber=os.environ["SOURCE_PHONE_NUMBER"],
            Attributes={"dialerContactId": req.contactId} if req.contactId else {},
        )
        return {"ok": True, "contactId": resp["ContactId"]}
    except Exception as e:
        raise HTTPException(502, f"Connect error: {e}")
```

Run:

```
uvicorn server:app --port 8787 --reload
```

4. Wiring the frontend

The dialer already calls `fetch(CALL_API, ...)` from `placeRealCall()` in `index.html`. `CALL_API` defaults to `/api/call`. Two ways to connect:

1. Serve index.html from the same origin as the backend. Add this to server.js:

   ``javascript import path from 'path'; import { fileURLToPath } from 'url';
   const __dirname = path.dirname(fileURLToPath(import.meta.url));
   app.use(express.static(path.join(__dirname, '..'))); // serve repo root

   Then open `http://localhost:8787/index.html`.

2. Run the static file from somewhere else (Vite, file://, S3) and set:

   ``html <script>window.CALL_API = 'http://localhost:8787/api/call';</script>

   right above the existing inline script in `index.html`.

If the backend is down or returns an error, the dialer's `placeRealCall()` catches the failure and falls back to the simulated call UI - so the app stays usable during development.

---

5. Verifying end-to-end

1. Start the backend.
2. Log into the Connect CCP as the agent and set status to Available.
3. Open the dialer, load sample contacts, edit one of them to use a real phone number you control, then click Call.
4. Expected: the CCP rings, you accept, the customer phone rings, and audio bridges.
5. Check Connect -> Metrics & quality -> Contact search in the AWS console - the call appears with the dialerContactId attribute attached.

Common failures:

Symptom	Likely cause
`AccessDeniedException`	IAM policy missing `connect:StartOutboundVoiceContact`, or wrong instance ARN
`InvalidRequestException: SourcePhoneNumber`	Phone number not E.164, or not claimed by this instance
Call places but immediately drops	Contact flow not published, or queue has no available agents
CCP does not ring	Agent status is "Offline", or browser mic permission denied
CORS error in browser console	Backend not running, or `cors()` middleware missing

---

6. Optional: embed the CCP inside the dialer (Streams SDK)

If you want the agent to take calls from inside the dialer UI itself (no separate CCP tab), add Amazon Connect Streams:

```
<script
src="https://unpkg.com/amazon-connect-streams@2.18.3/release/connect-streams-min.js"></scr
ipt>
<div id="ccp-container" style="display:none;"></div>
<script>
  connect.core.initCCP(document.getElementById('ccp-container'), {
    ccpUrl: 'https://YOUR-INSTANCE-ALIAS.my.connect.aws/ccp-v2/',
    loginPopup: true,
    softphone: { allowFramedSoftphone: true },
  });
  connect.contact(contact => {
    contact.onConnecting(() => console.log('Connecting to customer'));
    contact.onConnected(() => console.log('Customer on the line'));
    contact.onEnded(() => console.log('Call ended'));
  });
</script>
```

Then the embedded (hidden) CCP handles the WebRTC audio while the dialer UI shows your custom call overlay.

---

7. Project layout codex should produce

```
dialer/
+-- index.html                  (already exists - the dialer UI)
+-- AWS_CONNECT_SETUP.pdf       (this guide)
+-- backend/
    +-- package.json
    +-- .env                    (gitignored)
    +-- .env.example
    +-- server.js
```

Add `.env` and `node_modules/` to `.gitignore`.

---

8. Reference values to fill in

Before running:

- AWS_REGION - region of your Connect instance (e.g. us-east-1)
- CONNECT_INSTANCE_ID - UUID from the instance ARN
- CONNECT_CONTACT_FLOW_ID - UUID of the published Outbound-Simple flow
- SOURCE_PHONE_NUMBER - claimed Connect number in E.164 (+15558675309)
- AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY - from the dialer-backend IAM user

That is everything required to take the simulated dialer to a real phone-ringing outbound caller.